

AI-Powered Enterprise Operations and Monitoring Platform

Workflow Overview:

Backend (app.py):

- Input: Reads uploaded CSV files (System, Network, Login).
- Processing:
 - Selects random rows to create unique "Live Data" for every session.
 - Runs AI Models (.pkl files) to detect anomalies.
 - Logic Centre: Assigns Severity (Critical, High, Moderate, Low) based on specific Types of attacks (e.g., "DDoS" = Critical).
- Output: Sends a JSON response containing metrics and anomaly details to the Frontend.

Frontend (React):

- *UploadPage.jsx*: Handles file selection and sends them to the backend.
- *AnalysisContext.jsx* (The Engine): Receives the JSON data. It runs a Global Timer that releases data points one by one (every 1.5s) to simulate a live feed. This state is shared across the entire app.
- *Dashboard.jsx & Pages*: Listen to this shared "Live Feed" and update graphs/tables simultaneously.

File Control Guide:

Backend (Server Logic):

Location: backend/

File	Controls	Common Edits
app.py	The Brain. Controls file processing, AI prediction, and data logic.	• Attack Names: Edit NETWORK_INCIDENTS list to add new attacks (e.g., "Ransomware"). • Severity Rules: Edit SYSTEM_SEVERITY_MAP to change if "High CPU" is considered "High" or "Moderate".

Frontend (Logic & State):

Location: *src/context/*

File	Controls	Common Edits
AnalysisContext.jsx	The Heartbeat. Controls the "Live Simulation" timer and global data storage.	• Speed: Change the 1500 (ms) value to make graphs update faster or slower. • History Length: Change .slice(-19) to show more or fewer points on the charts.

Frontend: Pages (The Views):

Location: *src/pages/*

File	Controls	Common Edits
Dashboard.jsx	Main Screen. Aggregates all widgets (Stat Cards, Panels, Alert Table).	• Layout: Reorder how the panels appear. • Titles: Change the main dashboard header text.
SystemPage.jsx NetworkPage.jsx LoginPage.jsx	Individual Tabs. The full-screen views for specific monitors.	• Connection: Ensure they use liveData (from Context) to sync with the dashboard. • Page Headers: Change the specific titles of these pages.

File	Controls	Common Edits
UploadPage.jsx	Upload Screen. The file input interface.	• Loading Text: Change messages like "Running Isolation Forest...".

Frontend: Components (The Widgets):

Location: src/components/

File	Controls	Common Edits
Sidebar.jsx	Navigation Menu. The left-hand sidebar links.	• Menu Links: Add/Remove links (e.g., "Settings"). • Branding: Change the "AI Sentinel" text or logo.
StatCard.jsx	Top Cards. The 4 summary boxes (Total Anomalies, etc.).	• Icons: Swap the Lucide icons (Shield, Activity, etc.). • Colors: Change the background colors (Blue, Red, Purple).
AlertsTable.jsx	Main Table. The "Active Security Alerts" list is at the bottom.	• Badge Colours: Edit getSeverityBadge to change colours (e.g., make "Low" Blue instead of Green). • Columns: Add/Remove table columns.
SystemMonitoringPanel.jsx	System Widget. The Chart + Mini Table for System data.	• Chart Fill: Change the Blue/Purple area colours. • Mini-Table Rows: Change .slice(0, 10) to show more history.

File	Controls	Common Edits
NetworkMonitoringPanel.jsx	Network Widget. The Chart + Mini Table for Network data.	• Line Colours: Change the Red (Attacks) vs Blue (Normal) line colours.
LoginMonitoringPanel.jsx	Login Widget. The Chart + Mini Table for Login data.	• Line Colours: Change the Red (Suspicious) vs Green (Normal) line colours.

Tech Stack used in this project:

Backend (Server-Side)

- Programming Language: Python 3.10+
- Web Framework: Flask (Lightweight WSGI web application framework)
- Machine Learning Libraries:
 - scikit-learn (For Isolation Forest and Random Forest algorithms)
 - joblib (For loading pre-trained .pkl model files)
 - pandas (For data manipulation and reading CSV files)
 - numpy (For numerical operations and handling array data)
- API Handling:
 - flask-cors (To allow the React Frontend to communicate with the Python Backend)
 - werkzeug (For secure file handling and uploads)

Frontend (Client-Side)

- Library: React.js (JavaScript Library for building UIs)
- Styling: Tailwind CSS (Utility-first CSS framework for rapid UI development)
- State Management: React Context API (AnalysisContext) for managing global "Live" simulation data.

- Routing: react-router-dom (For navigation between Dashboard, System, Network, and Login pages)
- Data Visualization: recharts (For the responsive Line and Area charts)
- Icons: lucide-react (For modern SVG icons like Shield, User, Server)
- HTTP Client: axios (For sending file uploads to the backend)

AI/ML Models (The "Sentinel" Engine)

- Anomaly Detection: Isolation Forest (Unsupervised learning to find outliers)
- Classification: Random Forest Classifier (Supervised learning to classify specific attack types)
- Models Used:
 - *isolation_forest_system_model.pkl*
 - *random_forest_incident_model.pkl*
 - *network_intrusion_model.pkl*
 - *login_anomaly_iso_model.pkl*
 - *login_anomaly_rf_model.pkl*

Data Flow

1. **Input:** User uploads 3 CSV files (System, Network, and Login logs).
2. **Processing:** Python Backend reads random samples (memory-safe), runs them through the AI models, and assigns Severity/Type logic.
3. **Transport:** Data is sent as JSON objects via a REST API (POST /analyse).
4. **Simulation:** React Frontend (AnalysisContext) receives the data and runs an setInterval timer to "feed" the data point-by-point to the graphs, simulating a live, real-time stream.

Prerequisites (Install these first)

1. Python (v3.10 or newer): [Download Here](#)
 - *Note during install:* Check the box "Add Python to PATH".
2. Node.js (v18 or newer): [Download Here](#)
 - This includes npm (Node Package Manager) which is required for the frontend.

3. VS Code: (Optional but recommended code editor).

Step 1: Project Setup:

1. Copy your project folder (AIME) to the laptop.
2. Open the folder in **VS Code**.
3. Open the **Terminal** in VS Code.

Step 2: Backend Setup (Python)

1. Navigate to the backend folder:
~ cd backend
2. Create a Virtual Environment (keeps libraries isolated):
~ python -m venv venv
3. Activate the Virtual Environment:
 - a. Windows:
~ venv\Scripts\activate
4. **Install Required Libraries:** Run this command to install the specific versions used in your code:
~ pip install flask flask-cors pandas numpy scikit-learn joblib werkzeug

Library Version Breakdown:

- Flask: Web framework.
- flask-cors: Allows React to talk to Python.
- pandas: For reading CSV files.
- numpy: For math operations.
- scikit-learn: For the AI models (Isolation Forest/Random Forest).
- joblib: For loading the .pkl model files.

Step 3: Frontend Setup (React)

- Open a NEW Terminal (keep the Python one running or split the terminal).
- Navigate to the frontend folder:
~ cd frontend
- **Install Dependencies:** This reads your package.json file and installs everything automatically.
~ npm install

Key Libraries Installed:

- react, react-dom: Core React framework.
- react-router-dom: For navigating between Dashboard/System/Network pages.
- recharts: For the Line and Area charts.
- lucide-react: For the icons (Shield, User, etc.).
- axios: For sending files to the backend.
- tailwindcss, postcss, autoprefixer: For styling.

Step 4: Running the Project

You need two terminals running at the same time.

Terminal 1 (Backend):

~ cd backend

~ python app.py

Terminal 2 (Frontend):

~ cd frontend

~ npm run dev